Loco Pilot Analysis

Scalability, Data Findings and Cost Plan

Date: 2026-10-27 | Covers how the current code works, then the discussion from the throughput question onward (capacity, data inspection, loco scenarios, duration and camera findings).

Legend: [MEASURED] = observed directly on the data. [ESTIMATE] = calculated from stated assumptions, not benchmarked. [ASSUMPTION] = needs confirmation.

1. How the current system works (main3.py)

A FastAPI web app. The user uploads one video in the browser; the app analyses it and returns a results page (one card per parameter with status, confidence, time and evidence image) and a downloadable PDF report. It processes one video at a time.

1.1 Parameters detected: 9

#	Parameter	Main signals used
1	Crew Alertness	Eye closure share (PERCLOS), head droop (pitch below -25 deg), yawning. Score = 0.5 x eyes + 0.3 x droop + 0.2 x yawn -> HIGH / MODERATE / LOW
2	Crew Distraction	Head turned away from the normal direction (yaw more than 30 deg), gaze direction, body attention signal
3	Crew Doing Writing Work	Book/paper detected, hand and gaze down, paper-region signal, sustained for several seconds
4	Crew Taking Nap / Micro-sleep	Consecutive closed-eye frames, head droop, long mouth-open/yawn
5	Crew Using Mobile	YOLO 'cell_phone' detection held over consecutive frames
6	Crew Packing Belongings	YOLO backpack / handbag / suitcase detection with activity
7	Crew Leaving Place	Person count drops to zero or a tracked person disappears for a run of frames
8	Crew Operating Horn Lever/Switch	Custom YOLO class 'horn_active'
9	LP Applying A9/SA9 After Halt	Custom YOLO class 'brake_applied'

1.2 Models and components

Component	What it does
YOLOv8s (general)	Detects 7 object classes: person, cell phone, backpack, handbag, suitcase, book, laptop
brake_horn_detection.pt (custom YOLO)	4 classes: horn_active, horn_inactive, brake_applied, brake_released
MediaPipe Face Landmarker	Per-face eye aspect ratio, mouth aspect ratio (yawn), head pose (yaw/pitch), gaze direction, blendshapes
Qwen2.5-VL-7B (4-bit)	Looks at selected images of a candidate event and returns a JSON verdict; used to confirm or reject
ffmpeg (optional)	Repairs broken H.264 metadata by stream copy before decoding

1.3 What happens after a video is uploaded

Step	What happens
0. Upload	The video is saved and the runtime folders (frames, annotated, detections, profiles) are reset.

Step	What happens
1. Pre-process	ffmpeg stream-copy repairs the video metadata. If ffmpeg is missing it is skipped silently and the original file is used.
2. Frame extraction	One frame every 4 s (decord if available, else OpenCV). If that would exceed 200 frames the interval is widened so there are at most 200. The last 8 s are sampled every second. Frames are kept in memory.
3. Local computer vision	Both YOLO models run on all frames in batches of 32. MediaPipe then analyses faces in 4 parallel threads. Face metrics are smoothed over 3 frames and persons are tracked across frames (IoU matching).
4. Rule engine	Frame-level signals are turned into the 9 parameters, each with a status, confidence and time. Thresholds include: mobile 0.62, writing 0.48, packing 0.62, leaving 0.65, nap 0.62, distraction 0.58, horn 0.55, brake 0.55.
5. Event candidates	Runs of suspicious frames become candidate events with a local score. For writing, nap, alertness and distraction, a dense 6 s window at 1 frame/s is added around the event.
6. VLM verification	Up to 8 candidate events are sent to Qwen (slots: attention state 3, nap 1, writing 2, mobile 1, then leaving/horn/brake/packing). Each gets up to 6 context images at 448 px and a task-specific prompt. Output: decision yes/no/uncertain, confidence, subtype, reason (max 140 tokens, no sampling).
7. Fusion	Local score and VLM verdict are combined (for attention events 0.35 local + 0.65 VLM). Final statuses include HIGH, MODERATE, LOW, OBSERVED, NORMAL_OPERATION, UNCERTAIN. An annotated evidence frame is saved.
8. Output	Summary and recommendations are generated, a PDF report is built (fpdf2), a JSON profile with stage timings is written, and the results page is shown.

1.4 Observations about the current design

- The VLM is already used only on selected candidate events, not on every frame. Most of the compute is the CV stage and the fixed VLM load.
- Everything works on a single uploaded video held in memory; there is no queue, batch mode or multi-video handling.
- The Dockerfile does not install ffmpeg, so in the container the metadata-repair step is skipped and the original file is decoded. That matters for the raw Hikvision files (section 4).
- The camera tag and on-screen timestamp are not read. Wrong-camera or blocked footage goes through the same analysis.

2. Starting point and question

Short answer: the models can scale, but the current pipeline can't take this volume as it stands.

- The models scale: YOLO, MediaPipe and Qwen all run in batch on GPU workers, so more GPUs means more throughput.
- The pipeline does not: it is a single-video upload app with fixed sampling caps (200 frames, 8 VLM events per video), no queue or batch mode, and no handling of many videos in parallel.
- Measured by you: a 4:42 min video took 2:30 to analyse on a g4dn.2xlarge (T4 GPU, 16 GB VRAM), about 1.9x real time.
- Expected load (your first statement): 25-32 TB of video per day from 3-4 trains, videos of 1 h 47 min each.
- Questions: are the models scalable, how long would the full data take, what compute and cost are needed, and what must change because accuracy is low.

2.1 Why 1.9x real time does not extrapolate [MEASURED from code]

- The code samples at most MAX_FRAMES = 200 frames per video and at most VLM_MAX_EVENTS = 8

VLM checks per video.

- The 2:30 probably includes fixed costs (model loading). Per-video time is therefore roughly constant, not proportional to length.
- Consequence: a long video is analysed at about one frame every 32 s (1 h 47) or every 11 s (38 min). That is fast, but it misses short behaviours and is a main cause of low accuracy.

2.2 Cost model used [ESTIMATE]

- With proper sampling (1 frame per 4 s, about 1,600 frames per 1 h 47 video) one T4 needs roughly 6-10 min per video, i.e. about 4.5 GPU-minutes per video-hour. This must be benchmarked with the existing PipelineProfiler.
- Prices (verify on AWS): g4dn.2xlarge on-demand about $0.75/h, spot about $0.27/h (about 64% off).

3. The data that was inspected

Folder: D:\30274 cvvrs 03-02-2026 (4 drive folders). The name looks like a loco number plus CVVRS (Crew Voice and Video Recording System) [ASSUMPTION].

3.1 Facts [MEASURED]

- 6,673 hiv*.mp4 files, 1,791 GB total (about 1.8 TB). About 7,485 files overall including .pic, firmware and log files.
- Every video file is exactly 268,435,456 bytes (256 MiB): a fixed-size recorder chunk.
- Resolution (12 random files): 10 are 1280x720 (about 0.7-1.0 Mbps), 2 are 1920x1080 (about 2.6-3.3 Mbps). H.264, 25 fps.
- The files are NOT standard MP4. No ftyp/moov box, they start with proprietary bytes, so OpenCV/ffmpeg guess the stream. Decoding prints 'non-existing PPS' and 'decode_slice_header' errors and seeking fails.
- Each frame carries a burned-in overlay: date/time (MM-DD-YYYY) and a camera tag such as 'IPCamera 01'. Dates seen run from 12-04-2025 to 01-05-2026, consistent with a 40-50 day window.

3.2 Frames inspected [MEASURED, 4 files]

File	Camera / time	Scene	Res.
drive 1 / hiv01100	IPCamera 01, 12-29-2025 02:09	Cab, two crew members visible	720p
drive 2 / hiv00984	IPCamera 02, 01-05-2026 03:35	Empty cab in IR, lens droplets/blur	720p
drive 3 / hiv01701	IPCamera 06, 12-04-2025 00:21	Almost black IR, unusable	1080p
drive 4 / hiv00520	IPCamera 06, 12-08-2025 06:50	Outside track view, mostly blocked by smear	1080p

- Both 1080p files were Camera 06; both 720p files were Cameras 01/02, so resolution can hint at the camera.
- Only Camera 01 clearly showed crew behaviour. The others were empty, blocked or outside views.

4. The duration discrepancy

- OpenCV reported about 11-53 min per file (average about 38 min over 12 files). These come from guessed frame counts and are UNRELIABLE.
- You observed about 1 h 47 per file in a real player, which is more credible. A fixed 256 MiB chunk at 1 h 47 means about 0.33 Mbps average: low but possible.
- Possible risk: decord/cv2 in the pipeline may skip or mis-time frames on these raw Hikvision chunks, which could itself hurt accuracy.
- Resolution: export/convert properly (Hikvision player/SDK, or ffmpeg remux with -err_detect ignore_err), or read the on-screen timestamp at the start and end of a file.

5. Data volume: what the numbers say

Statement	Figure	Consistent?
Original: 25-32 TB/day	= 14-18 locos/day at 1.8 TB each	No
This folder	1.8 TB, 6,673 files, one loco, 40-50 days	Yes
3-4 locos/day (your clarification)	5.4-7.2 TB/day, 20,000-26,700 files/day	Yes

The 25-32 TB/day figure only works with far more locos per day or much higher bitrates than were inspected. Please confirm which figure is correct; all costs below scale with it.

5.1 Video-hours per loco

Scenario	Per file	Video-hours per loco	Per day (3-4 locos)
A: OpenCV durations	about 38 min	about 4,200 h	12,600-16,800 h
B: your observation	about 1 h 47	about 11,900 h	35,700-47,600 h

Scenario B means about 260 video-hours per day over 45 days, i.e. roughly 11 cameras recording 24 h [ESTIMATE; confirm camera count].

6. Compute, time and cost scenarios [ESTIMATE]

Scenario	GPU-h per loco	GPU-h per day	On-demand $/day	Spot $/day
A: 38 min files, all cameras	about 315	945-1,260	$710-950	$255-340
B: 1 h 47 files, all cameras	about 890	2,700-3,500	$2,000-2,600	$730-960
C: B, crew camera only (assumed 1 of 6 files)	about 150	445-590	$335-445	$120-160

6.1 T4 instances needed by turnaround target

Scenario	Same day	3 days	7 days
A	40-53	13-18	6-8
B	113-148	38-49	16-21
C	19-25	6-8	3-4

- Cost per day is about the same whatever the turnaround: a slower turnaround only means fewer instances running longer.
- Locos arrive about every 45 days, so same-day turnaround is probably not required. Multi-day turnaround on spot is the cheapest option.
- Earlier (before the camera finding) a driver-camera-only case was estimated at about one quarter of scenario A: 240-315 GPU-h/day, $180-240/day on-demand, $70-95/day on spot.

6.2 Storage and transfer

- Ingest: 5.4-7.2 TB/day is a sustained 0.5-0.7 Gbps. Copying one 1.8 TB loco over 1 Gbps takes about 4 h.
- S3 Standard (about $0.023/GB-month): each day's data costs about $125-165 per month. 90 days of raw video (about 650 TB) is about $15k/month.
- Recommendation: delete raw video after analysis and keep only flagged clips and reports (or a cold tier). Alternative: a small GPU server at the depot so the raw video never has to be uploaded.

7. Why this changes the plan

- Most footage may be unusable: empty cab, blocked lens, black IR, outside views. Only some cameras show crew.

- If about 1 in 6 files is crew-facing, daily GPU load drops 4-6x (scenario C). Needs verification by classifying every file.
- Accuracy: running the models on wrong-camera, blocked or wrongly decoded footage produces false alerts and misses. This may be a large part of the low accuracy, independent of the models.
- The burned-in timestamp gives true recording times without needing ffmpeg.

8. Accuracy changes required (in main3.py)

Ordered by likely impact:

- MAX_FRAMES = 200 (line 93): remove the cap. Sample every 2-4 s over the whole video, processed in chunks. Short behaviours (nap, phone, writing, distraction) are currently missed.
- VLM_MAX_EVENTS = 8 (line 107): a flat number per video is too few for long videos. Choose events by detector score and scale with duration.
- VLM_IMAGE_SIZE = 448 (line 109): the full cab frame makes faces tiny. Feed the VLM a face/person crop.
- Add temporal smoothing: require a behaviour to persist for N consecutive samples.
- GROUND_TRUTH_JSON = None: there is no measured accuracy. Build a labelled set from real footage and tune FUSION_THRESHOLDS per behaviour with precision/recall.
- Adapt models to cab data: fine-tune brake_horn_detection.pt, consider a LoRA or verifier-only use of Qwen, handle night/IR and tunnel lighting.
- Verify input decoding on the Hikvision chunks (section 4) and drop wrong-camera/blocked/black frames before analysis.

9. Conclusion

- Scalability: the models themselves scale (YOLO, MediaPipe and Qwen run in batch on GPU workers), but the current app is a single-video UI with fixed sampling caps, so it is not ready for this volume as is.
- Real volume: about 5.4-7.2 TB/day (3-4 locos x 1.8 TB per loco), not 25-32 TB/day, unless more locos or higher bitrates than inspected.
- Biggest cost drivers: the true file duration and how many files are crew-facing. Cost ranges from about $120/day (spot, crew camera only) to about $2,600/day (on-demand, all cameras at 1 h 47).
- Biggest accuracy drivers: sampling cap, few VLM events, small VLM input, no ground truth, and unusable or wrongly decoded footage.
- Recommended path: convert and verify decoding, add the camera/timestamp pre-filter, benchmark one file, fix sampling and VLM settings, then size the fleet from measured numbers using spot GPUs, queue-based workers and multi-day turnaround.